

A Detailed Related Works

A.1 Co-Clustering

Co-clustering seeks to reveal associations between row and column clusters through the simultaneous clustering of both dimensions in a data matrix. Initially, Hartigan [21] advocated co-clustering of variables and cases, enabling direct interpretation of the resulting clusters. Dhillon [13] established a connection between co-clustering and spectral graph partitioning. Regarding the spectral co-clustering algorithm, Kluger et al. [30] enabled different cluster numbers per dimension, Gao et al. [16] resolved higher-order problems, and Shi et al. [43] integrated constraint information. Employing information theory transforms co-clustering into an optimization problem, where different association measures yield distinct algorithms: [18, 21] utilizes a least-squares criterion, ITCC [14] maximizes mutual information, and BCC [2] optimizes the Bregman divergence. Numerous model-based and matrix factorization-based algorithms have also been proposed, as reviewed in detail in previous surveys [5, 53]. Recently, co-clustering techniques have accelerated improvements in recommender systems. For instance, Wu et al. [56] incorporated sampling techniques to accelerate recommendations, while Feng et al. [15] improved accuracy by limiting recommended items to the same cluster. However, these methods are typically employed to supplement information and enhance accuracy, with limited focus on addressing the memory constraints of embedding tables.

A.2 Bipartite Graph Clustering

Bipartite graph clustering is a method for uncovering underlying structural properties in diverse relationship networks. A straightforward approach is to transform the bipartite graph into a unipartite graph, thereby allowing the application of conventional graph clustering methods, e.g., [37, 51]. Moreover, projection-based methods [36, 45] generate a unipartite graph to yield higher-quality clusters, but this often results in a much denser graph structure. Approaches specifically developed for bipartite graph clustering include spectral clustering [13, 30], and statistical modeling [31, 62]. However, the aforementioned methods often involve considerable computational overhead, whereas modularity maximization [6, 28, 49] and label propagation [39, 46] methods are recognized for their computational efficiency. Its influence diffusion-based structure enables continuous and enhanced information exchange between users and items, making it particularly well-suited for recommendation systems. Recently, Wu et al. [57] innovatively exploited user-item interaction graphs to compress embedding tables for recommendation tasks. However, their approach simply applies modularity maximization, without adequately considering the unique data characteristics and clustering biases of recommender systems.

B Additional Experimental Details

B.1 Datasets Details

We conduct our experiments on four benchmark datasets, each widely utilized in recommendation research [23, 52, 57] and real-world scenarios. The datasets are detailed as follows:

- *Beauty*: A subset of Amazon product reviews, encompassing user interactions of beauty products.

- *Gowalla*: A check-in dataset capturing user location-sharing behaviors on the Gowalla platform.
- *Yelp2018*: Extracted from the 2018 Yelp Challenge, this dataset contains user interactions with local businesses.
- *AmazonBook*: A subset of Amazon product reviews, containing user interactions with books.

B.2 Parameter Settings

Table 7: Parameter setting in BACO

Parameter	<i>Beauty</i>	<i>Gowalla</i>	<i>Yelp2018</i>	<i>AmazonBook</i>
γ	0.13	7.57	5.50	4.73

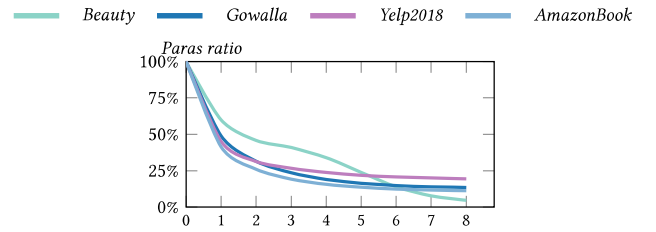


Figure 4: Embedding table parameters ratio of BACO versus iteration count.

In this section, we present the parameters not detailed in the main text. We utilize the Adam [29] optimizer with a learning rate of 0.001 and a mini-batch size of 1024, and an embedding dimension of 64 across all datasets. Training is conducted for up to 1000 epochs, with early stopping (patience of 50 epochs) and validation strategies employed to prevent overfitting.

Following the settings in [57], we assign half of the hash bins to the highest-frequency entities in both the Frequency Hashing [17] and Hybrid Hashing, and employ the user-item interaction graph as the feature in LSH [11]. In addition, CCE [50] and LEGCF [35] require dynamic updates, but for fairness, their sketching matrices are updated only in the first epoch, while hash labels for other methods are fixed before training. We retain the original implementations of infomap [41], BiMLPA [46], BRIM [38], as these methods inherently perform adaptive cluster detection without explicit control over the number of communities.

For a fair comparison, we adjust our parameter γ to match the target size of the embedding table, as in Table 7. As shown in Figure 4, our empirical study confirms that the parameter converges at an exponential rate. The parameter achieves around 20% compression ratio and tends to be stable after 5 iterations. Thereon, we fix the parameter T to 5.

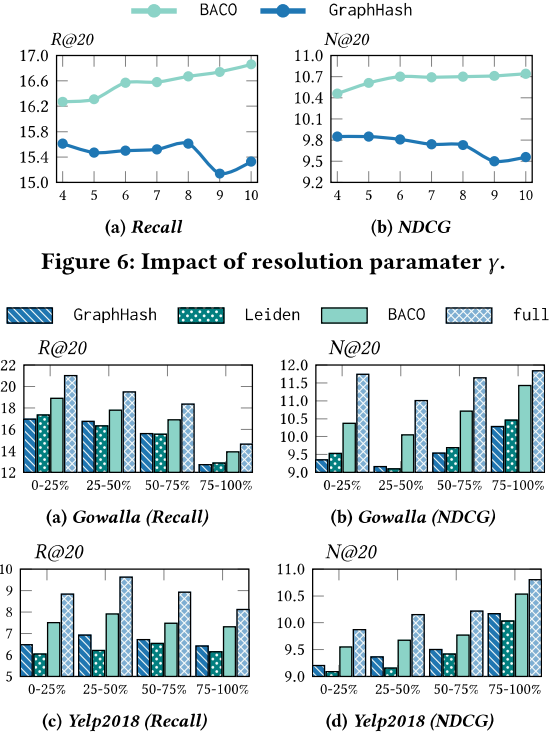
B.3 Evaluation Metrics

Given a set of $C = \{C_1, C_2, \dots, C_K\}$ of K disjoint co-clusters, each containing both users and items, we provide the formal mathematical definitions of the *averaged cross-cluster links* (ACCL) and the *Gini coefficient* in Figure 1 as follows:

$$\text{ACCL} = \frac{\sum_{C_k, C_\ell \in C} \sum_{u_i \in C_k, v_j \in C_\ell} B_{i,j}}{\binom{K}{2}}, \quad \text{Gini} = \frac{2}{K} \cdot \sum_{i=1}^K \left(\frac{i}{K} - \frac{\sum_{j=1}^i |C_j|}{\sum_{k=1}^K |C_k|} \right).$$

Table 8: Recommendation Performance ($k = 10$ or 50). Best results highlighted in blue and runner-up underlined.

Method	Beauty				Gowalla				Yelp2018				AmazonBook			
	R@10 $\uparrow$	N@10 $\uparrow$	R@50 $\uparrow$	N@50 $\uparrow$	R@10 $\uparrow$	N@10 $\uparrow$	R@50 $\uparrow$	N@50 $\uparrow$	R@10 $\uparrow$	N@10 $\uparrow$	R@50 $\uparrow$	N@50 $\uparrow$	R@10 $\uparrow$	N@10 $\uparrow$	R@50 $\uparrow$	N@50 $\uparrow$
Full Model	7.619	4.814	16.343	6.998	12.917	9.837	29.010	14.538	5.602	4.334	16.334	7.796	5.689	4.542	15.093	7.603
Random	3.054	1.874	7.555	3.009	6.368	5.015	15.097	7.588	3.048	2.414	9.055	4.375	1.677	1.445	4.679	2.448
Frequency Hashing [17]	2.926	1.763	7.637	2.945	5.852	4.597	14.331	7.108	3.071	2.451	9.262	4.482	1.459	1.243	4.095	2.127
Double Hashing [64]	2.872	1.717	7.035	2.779	6.491	5.133	15.172	7.705	3.170	2.504	9.542	4.587	1.626	1.374	4.393	2.304
Hybrid Hashing [64]	3.028	1.786	7.704	2.976	6.901	5.451	16.061	8.151	3.856	3.112	11.172	5.486	1.726	1.418	4.892	2.471
LSH [11]	3.613	2.179	8.720	3.470	6.481	5.191	14.244	7.511	2.858	2.255	8.274	4.033	2.287	1.899	6.260	3.212
CCE [50]	4.953	2.993	11.478	4.629	7.391	5.844	16.710	8.586	3.533	2.808	10.148	4.960	2.117	1.710	5.937	2.975
LEGCF [35]	4.989	2.914	12.391	4.754	5.413	4.046	13.230	6.319	2.747	2.071	8.060	3.795	-	-	-	-
GraphHash [57]	5.776	3.579	12.294	5.224	10.672	8.074	24.442	12.112	3.863	3.013	11.709	5.542	5.081	4.264	11.851	6.477
DoubleGraphHash [57]	3.883	2.439	9.307	3.826	8.711	6.720	19.949	10.007	3.245	2.529	9.708	4.635	3.708	2.778	9.221	4.519
LP [39]	5.465	3.415	12.162	5.094	8.026	6.253	19.144	9.481	3.358	2.636	10.291	4.868	2.278	1.830	6.435	3.181
Leiden [49]	5.540	3.446	12.165	5.108	10.753	8.203	24.245	12.152	3.898	3.000	11.764	5.536	5.174	4.300	11.927	6.513
EBMD [28]	5.443	3.290	11.961	4.949	7.061	5.501	16.816	8.352	3.080	2.407	9.628	4.525	2.182	1.780	6.052	3.058
infomap [41]	2.922	1.767	7.521	2.893	4.085	2.997	9.511	4.556	2.690	2.081	8.074	3.827	0.749	0.584	2.248	1.067
BiMLPA [46]	2.293	1.295	6.806	2.381	4.423	3.262	10.430	4.986	2.751	2.140	8.284	3.930	0.699	0.545	2.106	0.996
BRIM [38]	5.418	3.236	12.875	5.082	5.595	4.378	12.935	6.518	2.702	2.080	8.025	3.809	1.438	1.121	4.387	2.081
SCC [13]	5.920	3.640	13.072	5.456	9.667	7.467	21.880	11.016	4.167	3.224	12.246	5.832	3.622	2.909	9.869	4.941
SBC [30]	4.360	2.630	10.506	4.192	7.220	5.665	17.084	8.522	3.060	2.377	9.164	4.354	2.389	1.944	6.508	3.300
ITCC [14]	3.096	1.925	7.648	3.090	5.461	4.419	12.163	6.439	2.788	2.258	8.291	4.069	1.753	1.471	4.701	2.465
BACO	6.308	3.733	14.036	5.643	11.701	9.052	26.429	13.344	4.790	3.767	13.739	6.668	5.357	4.315	13.221	6.868
v.s. Baselines	+0.388	+0.093	+0.964	+0.187	+0.948	+0.849	+1.987	+1.192	+0.623	+0.543	+1.493	+0.836	+0.183	+0.015	+1.294	+0.355
v.s. Full Model	-1.311	-1.081	-2.307	-1.355	-1.216	-0.785	-2.581	-1.194	-0.812	-0.568	-2.596	-1.127	-0.332	-0.227	-1.872	-0.734

**Figure 5: Performance breakdown by test user frequency.**

C Additional Experimental Results

C.1 Recommendation Performance Evaluation

To comprehensively assess the performance of BACO, we additionally report Recall@ K and NDCG@ K for $K = 10$ and $K = 50$, using the same embedding table size as shown in Table 4. As shown in Table 8, BACO consistently outperforms all baseline approaches

across all datasets, with improvements of up to 1.987% in Recall and 1.192% in NDCG, achieving substantial improvements in both evaluation settings. Overall, these results demonstrate the substantial effectiveness of BACO under both strict and relaxed scenarios.

C.2 User Subgroup Evaluation

This experiment investigates the efficacy of algorithms across user groups, which are categorized by their activity frequency percentiles in the training data. In the Figure 5, we report the average metrics of each degree subgroup for both the top-performing baselines and the full model. All methods follow the trend of the full model and perform better with power users. Notably, BACO, which achieves the best overall performance, substantially mitigates the shortcomings of existing algorithms with respect to tail users. These observations indicate that substantial improvements can still be made for low-frequency users, and our framework provides a potential solution to address this challenge.

C.3 Parameter Analysis

In Figure 6, we vary γ in the range $[4, 10]$ on Gowalla, where the compression ratio parameter spans $[1/10, 1/5]$. BACO consistently surpasses GraphHash across all resolutions. Furthermore, increasing the resolution (with a corresponding increase in the embedding table size) leads to an improvement in Recall for BACO, whereas GraphHash fails to achieve further performance gains. Regarding NDCG, BACO maintains stable performance, while GraphHash demonstrates a decline, due to its coarse clustering strategy.

C.4 Clustering Result Analysis

We further examine the differences in clustering between BACO and strong baselines by analyzing cluster size distribution and embedding distance.

As illustrated in Figure 7, BACO exhibits a more heterogeneous cluster size distribution compared to GraphHash and Leiden, with

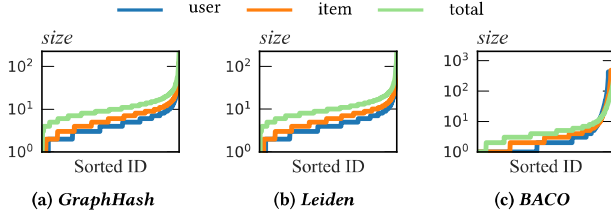


Figure 7: Cluster size distributions of GraphHash, Leiden, BACO.

a greater prevalence of both small and large clusters. This pattern arises because our method groups low-degree nodes into large clusters, facilitating mutual information sharing, while assigning high-degree nodes to smaller or singleton clusters to minimize conflicts. In contrast, other methods tend to produce clusters of more uniform size, thereby failing to fully leverage the inherent graph structure.

Table 9: Average distance of full embeddings and codebooks.

Method	Gowalla			Yelp2018		
	user	item	all	user	item	all
GraphHash	5.444	4.976	5.174	5.291	4.894	5.074
Leiden	5.401	4.951	5.141	5.449	4.970	5.188
SCC	5.405	4.872	5.097	5.038	4.794	4.905
BACO w/o SCU	5.069	4.909	4.977	5.180	4.847	4.998
BACO	4.854	4.980	4.927	4.751	4.982	4.877

Table 9 presents the average distances between the hashing embeddings and the full model embeddings for users, items, and the combined set. The results indicate that embeddings produced by BACO are more closely aligned with those of the full model, corroborating its superior performance as reported in Table 9. Moreover, the SCU strategy in BACO substantially reduces the user-side distance, with only minimal impact on item-side conflicts. Nevertheless, the overall improvement for both users and items is more significant.

C.5 Additional Large-scale Datasets

Table 10: Summary statistics about large-scale datasets.

Dataset	#Users	#Items	#Interactions	Density
MovieLens	200,808	65,032	20,228,336	0.155%
SteamGame	2,567,538	15,474	7,793,069	0.020%

Table 11: Performance comparison on large-scale datasets.

Method	MovieLens			SteamGame		
	Param↓	R@20↑	N@20↑	Param↓	R@20↑	N@20↑
Full Model	17.0M	25.980	20.294	165.3M	8.004	3.752
GraphHash	2.26M	20.631	14.669	22.6M	6.354	2.821
Leiden	2.26M	20.878	14.778	22.6M	6.439	2.830
BACO	2.19M	21.362	14.802	21.7M	6.912	3.111
v.s. Baselines	-	+0.484	+0.024	-	+0.473	+0.281
v.s. Full Model	-87.1%	-4.618	-5.492	-86.9%	-1.092	-0.641

We further evaluate BACO on *MovieLens* and *SteamGame*, two large-scale datasets with 20M interaction edges and 2M nodes, respectively, as shown in Table 10. We select the three optimal baselines, namely GraphHash, Leiden, and SCC, for performance evaluation on large-scale datasets. Note that since SCC relies on the costly SVD technique, it fails to finish running within 10 hours. Therefore, we only present the results of GraphHash, Leiden, and BACO.

As shown in Table 11, BACO overall outperforms all the baselines with fewer embedding table parameters. Compared to the best baselines, BACO achieves improvement of over 0.4% in Recall@20 across both datasets. In terms of speed, BACO saves 25% of the running time relative to Leiden on *MovieLens* data. On *SteamGame*, BACO runs 1.5s longer than Leiden, yet it achieved relative improvements of 7.3% and 9.9% in Recall@20 and NDCG@20, respectively. The reason lies in our compact weighting schemes, which are tailored to the recommendation task.